

TELECOMMUNICATIONS CUSTOMER CHURN PREDICTION

Farook Ahmed Ali Shaik

INTRODUCTION

Q1: Describe the dataset. What is the dataset about?

The dataset is a customer churn prediction dataset consists **7043 rows** and **21 columns** chosen from kaggle dataset which is from a telecommunications company, and this dataset contains information about customers' demographics, service usage, and churn status.

VARIABLES ARE:

Categorical data: gender, Partner, Dependents, PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod, Churn, SeniorCitizen.

Integer data: tenure

Double/Float: MonthlyCharges, ToalCharges

String: CutomerID

Q2: Explain briefly on the target population, sampling strategies, potential bias, etc.

The target population included of customers for the telecommunications company and possibly the data might have collected through random sampling method and also might or might not include potential biases like overrepresentation or underrepresentation of specific groups, like age, gender, or contract type, which could affect churn predictions.

Also expecting few of the variables might undergo skewness interms of the data distribution that needs to be addressed if present.

Q3: Identify and define prediction problem(s).

The main prediction problem is to predict whether a customer will **churn (yes/no)**. The target variable is **churn** and the goal is to use the variables/features from the dataset to develop an efficient predictive model.

Q4: Discuss how to split the data into training and test sets.

The data is split into training, validation, and test sets as **(70%,15%,15%)** proportions respectively. Using the createDataPartition function from the caret package, the data is split stratified to ensure

that both the training and test sets maintain a similar distribution of the **Churn** variable (i.e., the proportion of churn and non-churn customers). This approach helps to address potential class imbalance.

CODE

```
library(tidyverse)
library(dplyr)
library(ggplot2)
library(corrplot)
library(rsample)
library(caret)
library(e1071)
library(reshape2)
library(cowplot)
library(gridExtra)
library(randomForest)
library(pROC)
library(xgboost)
library(randomForest)

setwd("D:\\.4 SEMESTERS KSU\\2_FALL 2024 KSU\\Statistical Learning\\Final Project SL")

data <- read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

df <- data

#displaying the first 5 columns of the df data
head(df,3)

## # A tibble: 3 x 21
##   customerID gender SeniorCitizen Partner Dependents tenure PhoneService
##   <chr>      <chr>      <dbl> <chr>    <chr>      <dbl> <chr>
## 1 7590-VHVEG Female          0 Yes     No          1 No
## 2 5575-GNVDE Male          0 No      No         34 Yes
## 3 3668-QPYBK Male          0 No      No          2 Yes
## # i 14 more variables: MultipleLines <chr>, InternetService <chr>,
## #   OnlineSecurity <chr>, OnlineBackup <chr>, DeviceProtection <chr>,
## #   TechSupport <chr>, StreamingTV <chr>, StreamingMovies <chr>,
## #   Contract <chr>, PaperlessBilling <chr>, PaymentMethod <chr>,
## #   MonthlyCharges <dbl>, TotalCharges <dbl>, Churn <chr>
```

```
summary(df)
```

```
##   customerID      gender  SeniorCitizen  Partner
## Length:7043    Length:7043    Min.    :0.0000  Length:7043
## Class :character  Class :character  1st Qu.:0.0000  Class :character
## Mode  :character  Mode  :character  Median :0.0000  Mode  :character
##                                     Mean  :0.1621
##                                     3rd Qu.:0.0000
##                                     Max.   :1.0000
##
##   Dependents      tenure  PhoneService  MultipleLines
## Length:7043    Min.    : 0.00  Length:7043    Length:7043
## Class :character  1st Qu.: 9.00  Class :character  Class :character
## Mode  :character  Median :29.00  Mode  :character  Mode  :character
##                                     Mean  :32.37
##                                     3rd Qu.:55.00
##                                     Max.   :72.00
##
##   InternetService  OnlineSecurity  OnlineBackup  DeviceProtection
## Length:7043    Length:7043    Length:7043    Length:7043
## Class :character  Class :character  Class :character  Class :character
## Mode  :character  Mode  :character  Mode  :character  Mode  :character
##
##
##
##   TechSupport  StreamingTV  StreamingMovies  Contract
## Length:7043    Length:7043    Length:7043    Length:7043
## Class :character  Class :character  Class :character  Class :character
## Mode  :character  Mode  :character  Mode  :character  Mode  :character
##
##
##
##
##   PaperlessBilling  PaymentMethod  MonthlyCharges  TotalCharges
## Length:7043    Length:7043    Min.    : 18.25  Min.    : 18.8
## Class :character  Class :character  1st Qu.: 35.50  1st Qu.: 401.4
## Mode  :character  Mode  :character  Median : 70.35  Median :1397.5
##                                     Mean  : 64.76  Mean  :2283.3
##                                     3rd Qu.: 89.85  3rd Qu.:3794.7
##                                     Max.   :118.75  Max.   :8684.8
##                                     NA's    :11
##
##   Churn
## Length:7043
## Class :character
```

```
## Mode :character
##
##
##
##
```

we can observe there are **NA's** present in **TotalCharges** variable which is going to get addressed soon in next code chunks and we have to convert the **SeniorCitizen** variable to categorical type which is supposed to be.

CONVERTING CATEGORY TO FACTORS

Categorical Variables need to get converted to factors for proper statistical handling and also ensures correct encoding and treatment in models.

Binary Conversion, SeniorCitizen variable which is stored as 0's & 1's need to get converted to a factor with meaningful labels.

```
selected_columns <- c(
  "gender", "Partner", "Dependents", "PhoneService", "MultipleLines",
  "InternetService", "OnlineSecurity", "OnlineBackup", "DeviceProtection",
  "TechSupport", "StreamingTV", "StreamingMovies", "Contract",
  "PaperlessBilling", "PaymentMethod", "Churn"
)

# Apply conversion to factors
df[selected_columns] <- lapply(df[selected_columns], as.factor)

# converting SeniorCitizen variable to factor/categorical
df$SeniorCitizen <- factor(df$SeniorCitizen, levels = c(0, 1), labels = c("No", "Yes"))

glimpse(df)
```

```
## Rows: 7,043
## Columns: 21
## $ customerID      <chr> "7590-VHVEG", "5575-GNVDE", "3668-QPYBK", "7795-CF0CW~
## $ gender          <fct> Female, Male, Male, Male, Female, Female, Male, Femal~
## $ SeniorCitizen    <fct> No, No, No, No, No, No, No, No, No, No, No, No, No, N~
## $ Partner          <fct> Yes, No, No, No, No, No, No, No, Yes, No, Yes, No, Ye~
## $ Dependents       <fct> No, No, No, No, No, No, Yes, No, No, Yes, Yes, No, No~
## $ tenure           <dbl> 1, 34, 2, 45, 2, 8, 22, 10, 28, 62, 13, 16, 58, 49, 2~
## $ PhoneService     <fct> No, Yes, Yes, No, Yes, Yes, Yes, No, Yes, Yes, Yes, Y~
## $ MultipleLines    <fct> No phone service, No, No, No phone service, No, Yes, ~
```

```
## $ InternetService <fct> DSL, DSL, DSL, DSL, Fiber optic, Fiber optic, Fiber o~
## $ OnlineSecurity <fct> No, Yes, Yes, Yes, No, No, No, Yes, No, Yes, Yes, No ~
## $ OnlineBackup <fct> Yes, No, Yes, No, No, No, Yes, No, No, Yes, No, No in~
## $ DeviceProtection <fct> No, Yes, No, Yes, No, Yes, No, No, Yes, No, No, No in~
## $ TechSupport <fct> No, No, No, Yes, No, No, No, No, Yes, No, No, No inte~
## $ StreamingTV <fct> No, No, No, No, No, Yes, Yes, No, Yes, No, No, No int~
## $ StreamingMovies <fct> No, No, No, No, No, Yes, No, No, Yes, No, No, No inte~
## $ Contract <fct> Month-to-month, One year, Month-to-month, One year, M~
## $ PaperlessBilling <fct> Yes, No, Yes, No, Yes, Yes, Yes, No, Yes, No, Yes, No~
## $ PaymentMethod <fct> Electronic check, Mailed check, Mailed check, Bank tr~
## $ MonthlyCharges <dbl> 29.85, 56.95, 53.85, 42.30, 70.70, 99.65, 89.10, 29.7~
## $ TotalCharges <dbl> 29.85, 1889.50, 108.15, 1840.75, 151.65, 820.50, 1949~
## $ Churn <fct> No, No, Yes, No, Yes, Yes, No, No, Yes, No, No, No, N~
```

HANDLING MISSING VALUES USING MEDIAN IMPUTATION

For the missing values in the **TotalCharges** column (11 NA'S), I am going to use median imputation and I believe it is a good choice because for numerical features with a small number of missing values like 11 i believe median imputation works fine.

Also it is less sensitive to outliers compared to mean imputation. And not wanted to go for PMM imputation or other complex methods because it is not necessary so to use for less missing values.

CHECK FOR MISSING VALUES

```
colSums(is.na(df))
```

```
##      customerID      gender SeniorCitizen      Partner
##           0           0           0           0
##      Dependents      tenure  PhoneService MultipleLines
##           0           0           0           0
## InternetService OnlineSecurity OnlineBackup DeviceProtection
##           0           0           0           0
##      TechSupport      StreamingTV StreamingMovies      Contract
##           0           0           0           0
## PaperlessBilling PaymentMethod MonthlyCharges TotalCharges
##           0           0           0           11
##           Churn
##           0
```

So Clearly even we observed earlier there are few NA's are present in **TotalCharges** variable which are being addressed below.

```
# Handling missing values using median imputation
df$TotalCharges[is.na(df$TotalCharges)] <- median(df$TotalCharges, na.rm = TRUE)

sum(is.na(df))
```

```
## [1] 0
```

CHECK FOR CLASS DISTRIBUTION

```
table(df$Churn)
```

```
##
##    No    Yes
## 5174 1869
```

```
prop.table(table(df$Churn))
```

```
##
##           No           Yes
## 0.7346301 0.2653699
```

The proportions in the dataset show that a *significant imbalance is present between the No (73.5%) and Yes (26.5%) classes* in the Churn variable. So there's a high chance the train and test datasets may not reflect the original distribution of Churn, and will lead to biased models, especially for the minority class = Yes. So this issue is addressed below.

STRATIFIED DATA SPLIT INTO TRAIN, VALID, TEST DATA SETS

```
set.seed(123)
```

```
#createDataPartition from caret package addresses the imbalance issue by performing a stratified
train_index <- createDataPartition(df$Churn, p = 0.7, list = FALSE)
train_data <- df[train_index, ]
```

```
# Remaining data for validation and test sets
remaining_data <- df[-train_index, ]
```

```
valid_index <- createDataPartition(remaining_data$Churn, p = 0.5, list = FALSE)
valid_data <- remaining_data[valid_index, ]
test_data <- remaining_data[-valid_index, ]
```

```
# Checking the proportions in each split and they successfully train, valid, test data sets has equal proportions  
prop.table(table(train_data$Churn))
```

```
##  
##           No           Yes  
## 0.7345366 0.2654634
```

```
prop.table(table(valid_data$Churn))
```

```
##  
##           No           Yes  
## 0.7348485 0.2651515
```

```
prop.table(table(test_data$Churn))
```

```
##  
##           No           Yes  
## 0.7348485 0.2651515
```

STATISTICAL LEARNING STRATEGIES AND METHODS

Q1: Perform exploratory data analysis using the training set.

FOR NUMERICAL DATA:

Histograms/Density Plots: Examining individual distributions of numerical features in order to understand their spread, central tendency, and potential skewness.

Boxplots: To visualize the distribution and also to identify outliers if any like boxplots of numerical variables (*tenure*, *MonthlyCharges*, *TotalCharges*) and plotting boxplots conditioned with target variable to know its relationships with the target (e.g., churn or no churn).

Correlation Matrix and Correlation Coefficients: Calculating correlations between numerical variables to identify linear dependencies to address and avoid multicollinearity issue if present. And plotting a heatmap for visualization.

FOR CATEGORICAL DATA:

Bar Charts (Unconditioned): Analyzing the frequency distributions for categorical variables independently.

Bar Charts (Conditioned on Target Variable): To know how each categorical variable interacts with target variable, which might possibly can reveal significant patterns or predictors.

Cross Tabulation and Chi-Square Test: To know the dependency between categorical variables and the target variable which helps in assessing statistical significance and drop out the insignificant variables that have no impact in churn prediction.

Q2: Describe statistical learning approaches and strategies for feature engineering.

Feature Engineering Strategies: Feature Engineering is being performed after splitting the data into training, validation, and test sets because to prevent Data Leakage.

NUMERICAL FEATURES:

1. Creating two new features ServiceCount and tenurebin and apply for all the train, valid, and test datasets.

- **ServiceCount:** Sum of binary service indicators like PhoneService, MultipleLines, and etc.
- **Tenure:** Binning the tenure variable into categories.

2. Skewness: Finding SKewness and if present apply transformation/standardization like log transformation.

3. Handling Outliers: Using IQR method or Z- Score to address outliers if present.

4. Correlation Analysis: Plotting correlation matrix, correlation coefficients and is computed to identify multicollinearity among features and drop out highly correlated features to reduce redundancy as well.

5. Feature Selection: Selecting features with strong correlation to the target variable and also using domain knowledge and statistical methods to retain only the most relevant features.

CATEGORICAL FEATURES

The categorical features need to get converted to numerical way so that the models can perform better and provide better results.

1. Transformation: Using one hot encoding to convert categorical variables into numerical format.

2. Feature Selection: Using the tests like chi square test results to select significant predictors based on their association with the target variable.

This way by performing Feature selection reduces dimensionality, improves interpretability, and prevents overfitting, which is vital for generalizing to unseen data.

Statistical Learning Methods:

Techniques like logistic regression or decision trees can be applied to predict the churn for telco customer data. These models are suitable for binary classification problems and can incorporate both numerical and categorical features.

Where logistic regression model can work as baseline model and easily interpretable and gives a good head start to work on complex and powerful models.

Q3: Based on the conditions assumed by the statistical learning methods, discuss their applicability to the prediction problem.

Logistic regression or decision trees like random forest appear appropriate for this dataset also can handle both numerical and categorical data well.

1. Logistic Regression:

Assumes linear relationships between features and the target. Strong correlations or collinearity among predictors can degrade this model's performance, that's why careful feature selection is critical.

2. Tree-Based Models (e.g., Decision Trees, Random Forest):

Do not require feature scaling or encoding but could perform better if data is encoded and feature selection to improve computational efficiency.

Handles categorical variables natively and these tree models are robust to outliers. And if numerical data distributions are non-linear or complex, tree-based methods may outperform linear approaches.

Key Observation to look out for:

- One-hot encoding may lead to high-dimensional data for categorical variables and needs to be addressed if required, especially for logistic regression we can use of ridge or lasso regressions.

FEATURE ENGINEERING

Let's do feature engineering for train, valid, test data.

ServiceCount: Sum of binary service indicators like PhoneService, MultipleLines, etc.

Tenure: Binning the tenure variable into categories.

```
service_columns <- c("PhoneService", "MultipleLines", "InternetService", "OnlineSecurity", "OnlineBackup")

# creating the new ServiceCount column by adding all binary columns
train_data$ServiceCount <- rowSums(train_data[service_columns] == "Yes", na.rm = TRUE)

# creating tenure_bin with bins as a new column based on tenure variable that we have
train_data$tenure_bin <- cut(train_data$tenure,
                             breaks = c(0, 12, 24, 36, 48, 60, Inf),
                             labels = c("0-1 year", "1-2 years", "2-3 years", "3-4 years", "4-5 years", "5+ years"),
                             right = FALSE)
```

```

#head(train_data,3)

# VALIDATION DATA

valid_data$ServiceCount <- rowSums(valid_data[service_columns] == "Yes", na.rm = TRUE)

valid_data$tenure_bin <- cut(valid_data$tenure,
                             breaks = c(0, 12, 24, 36, 48, 60, Inf),
                             labels = c("0-1 year", "1-2 years", "2-3 years", "3-4 years", "4-5
                             right = FALSE)

#head(valid_data,3)

# TEST DATA

test_data$ServiceCount <- rowSums(test_data[service_columns] == "Yes", na.rm = TRUE)

test_data$tenure_bin <- cut(test_data$tenure,
                             breaks = c(0, 12, 24, 36, 48, 60, Inf),
                             labels = c("0-1 year", "1-2 years", "2-3 years", "3-4 years", "4-5
                             right = FALSE)

#head(test_data,3)

```

ANALYZE NUMERICAL FEATURES

```

numerical_variables <- sapply(train_data, is.numeric)
numerical_data <- train_data[, numerical_variables]

```

HISTOGRAMS

```

plot1 <- ggplot(numerical_data, aes(x = .data$tenure)) +
  geom_histogram(binwidth = 5, fill = "skyblue", alpha = 0.7) +
  labs(title = paste("Histogram of tenure"), x = "tenure", y = "Frequency") +
  theme_minimal()

```

```

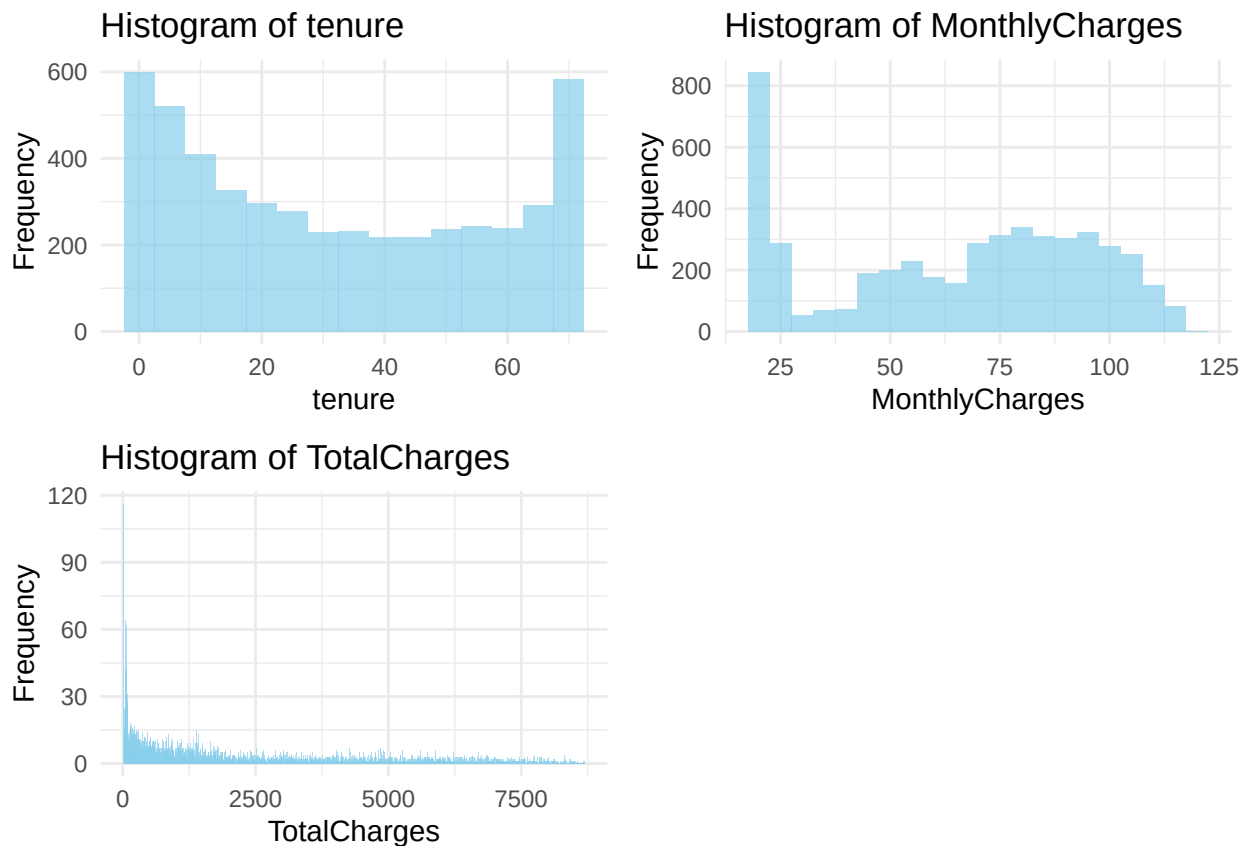
plot2 <- ggplot(numerical_data, aes(x = .data$MonthlyCharges)) +
  geom_histogram(binwidth = 5, fill = "skyblue", alpha = 0.7) +
  labs(title = paste("Histogram of MonthlyCharges"), x = "MonthlyCharges", y = "Frequency") +
  theme_minimal()

plot3 <- ggplot(numerical_data, aes(x = .data$TotalCharges)) +
  geom_histogram(binwidth = 5, fill = "skyblue", alpha = 0.7) +
  labs(title = paste("Histogram of TotalCharges"), x = "TotalCharges", y = "Frequency") +
  theme_minimal()

combined_plot <- plot_grid(plot1, plot2, plot3, ncol = 2)

combined_plot

```



INTERPRETATION:

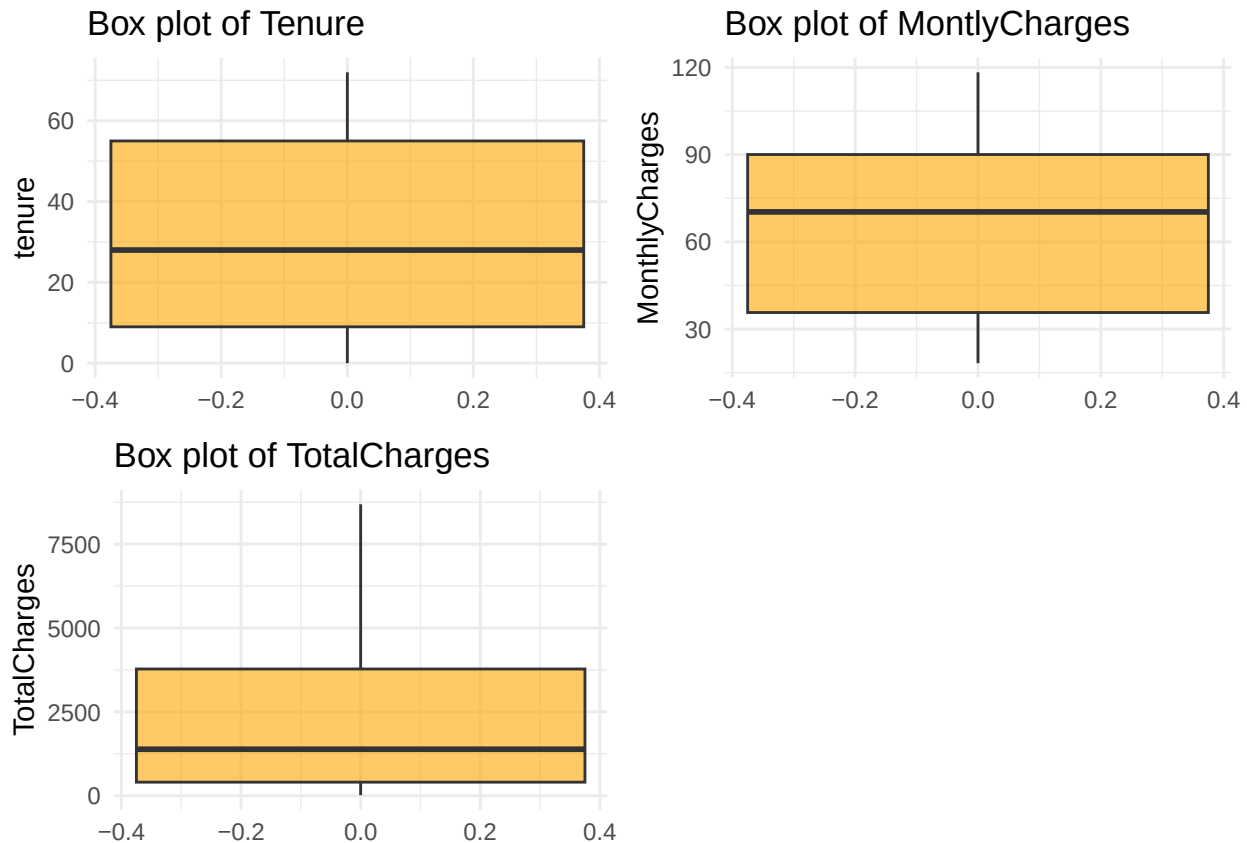
Tenure: Peak areas indicate most common tenure lengths and ranges. Concentration of customers in specific tenure are close extremes.

MontlyCharges: Reveals spread of monthly billing amounts and most common price points are from 75-100f monthly charges.

TotalCharges: shoes cumulative charges over customer lifetime and reveals potential high value vs low value customer segments where high value customer seems to be very low compared to low totalcharges people.

BOXPLOTS

```
plot1 <- ggplot(numerical_data,aes(y= .data$tenure))+  
  geom_boxplot(fill="orange", alpha=0.6)+  
  labs(title = "Box plot of Tenure") +  
  theme_minimal()  
  
plot2 <- ggplot(numerical_data,aes(y=.data$MonthlyCharges))+  
  geom_boxplot(fill="orange", alpha=0.6)+  
  labs(title = "Box plot of MontlyCharges") +  
  theme_minimal()  
  
plot3 <- ggplot(numerical_data,aes(y=.data$TotalCharges))+  
  geom_boxplot(fill="orange", alpha=0.6)+  
  labs(title = "Box plot of TotalCharges") +  
  theme_minimal()  
  
combined_plot <- plot_grid(plot1, plot2, plot3, ncol = 2)  
  
combined_plot
```



INTERPRETATION:

The *Tenure* and *MonthlyCharges* have close to evenly distribution but *totalcharges* distribution tells that most of the data is concentrated to lower end and we see in above histogram people paying high amounts are less.

```
#Checking the skewness to see the how the distribution looks like
skewness_results <- sapply(numerical_data, skewness)
skewness_results
```

```
##          tenure MonthlyCharges  TotalCharges  ServiceCount
##      0.2570418      -0.2244766      0.9667273      0.3185407
```

INTERPRETATION:

Skewness in data distribution becomes a potential problem when its goes beyond absolute value one.

tenure: 0.2570418, slightly positive skewed but acceptable.

ServiceCount: 0.3185407, slightly positive skewed but acceptable.

MonthlyCharges: -0.2244766, slightly negative skewed but acceptable.

TotalCharges: 0.9667273, Positively skewed and very close to 1, and has the highest skewness among the four and it has tendency towards lower values with a longer tail on the right side of the distribution. and needs to get addressed.

OUTLIERS DETECTION USING IQR

Inter Quartile Range method is chosen as it is less sensitive to skewness because it focuses on the spread of the middle 50% of the data and doesn't rely on the mean or standard deviation, which are sensitive to outliers.

IQR METHOD

```
for (variable in colnames(numerical_data)) {  
  Q1 <- quantile(numerical_data[[variable]], 0.25, na.rm = TRUE)  
  Q3 <- quantile(numerical_data[[variable]], 0.75, na.rm = TRUE)  
  IQR <- Q3 - Q1  
  lower_bound <- Q1 - 1.5 * IQR  
  upper_bound <- Q3 + 1.5 * IQR  
  
  outliers <- which(numerical_data[[variable]] < lower_bound | numerical_data[[variable]] > upper_bound)  
  
  if (length(outliers) > 0) {  
    cat(paste("Outliers detected in", variable, ":", length(outliers), "\n"))  
  }  
}  
  
outliers
```

```
## integer(0)
```

Interestingly we do not found any outlier through IQR method.

LETS PLOT BOX PLOT CONDITIONING WITH TARGET VARIABLE

```
# Box plot for tenure by Churn  
plot1 <- ggplot(train_data, aes(x = Churn, y = tenure, fill = Churn)) +  
  geom_boxplot() +  
  theme_minimal() +  
  labs(title = "Box plot of Tenure by Churn Status", x = "Churn Status", y = "Tenure (Months)")  
  theme(legend.position = "none")
```

```

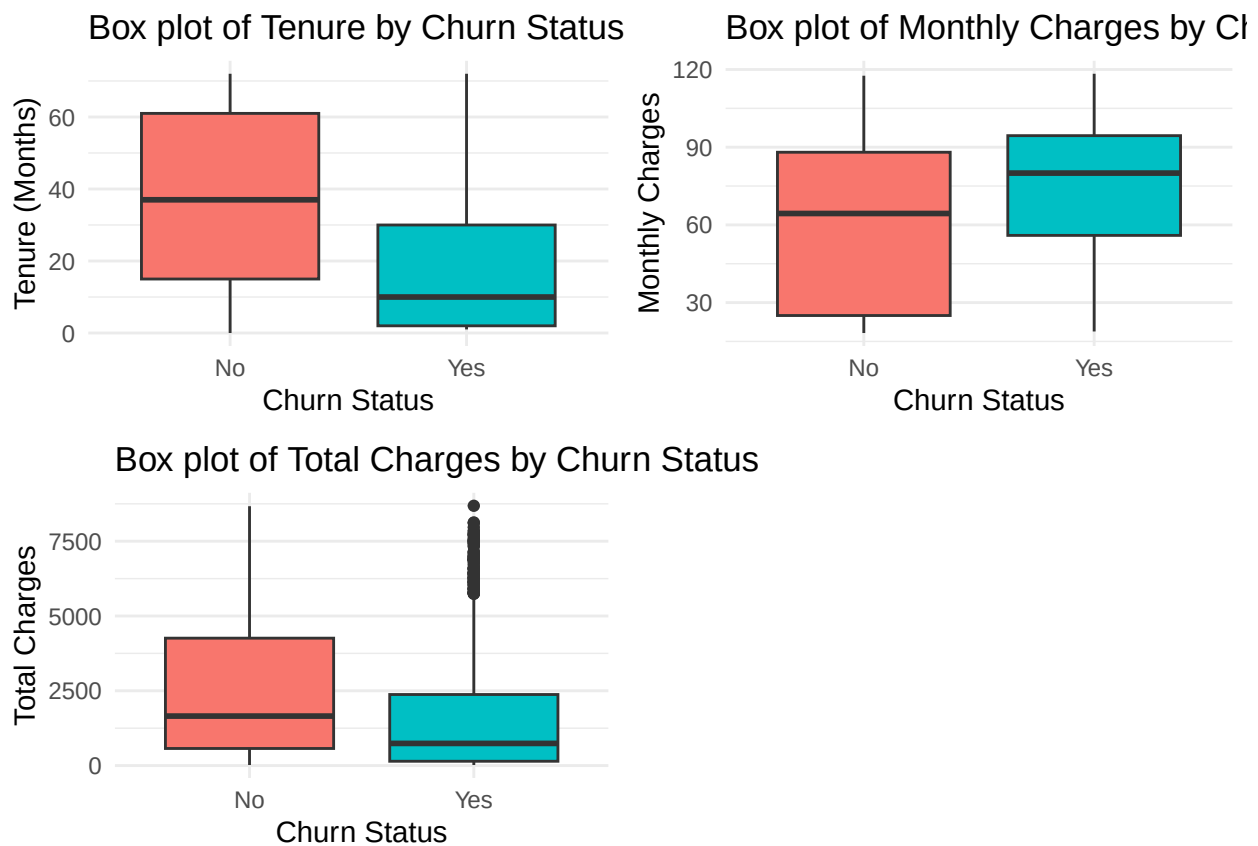
# Box plot for MonthlyCharges by Churn
plot2 <- ggplot(train_data, aes(x = Churn, y = MonthlyCharges, fill = Churn)) +
  geom_boxplot() +
  theme_minimal() +
  labs(title = "Box plot of Monthly Charges by Churn Status", x = "Churn Status", y = "Monthly C
  theme(legend.position = "none")

# Box plot for TotalCharges by Churn
plot3 <- ggplot(train_data, aes(x = Churn, y = TotalCharges, fill = Churn)) +
  geom_boxplot() +
  theme_minimal() +
  labs(title = "Box plot of Total Charges by Churn Status", x = "Churn Status", y = "Total Charg
  theme(legend.position = "none")

combined_plot <- plot_grid(plot1, plot2, plot3, ncol = 2)

combined_plot

```



```
#summary(train_data$TotalCharges)
```

Suprisingly we found outliers for **TotalCharges** variable when conditionnig with **target** variable, The discrepancy might have arised because earlir we looped through entire dataset without conditioning on Churn target variable to calculate outliers. But When we filtered by Churn = Yes in the box plot, the distribution of TotalCharges caught new outliers specific to this subset.

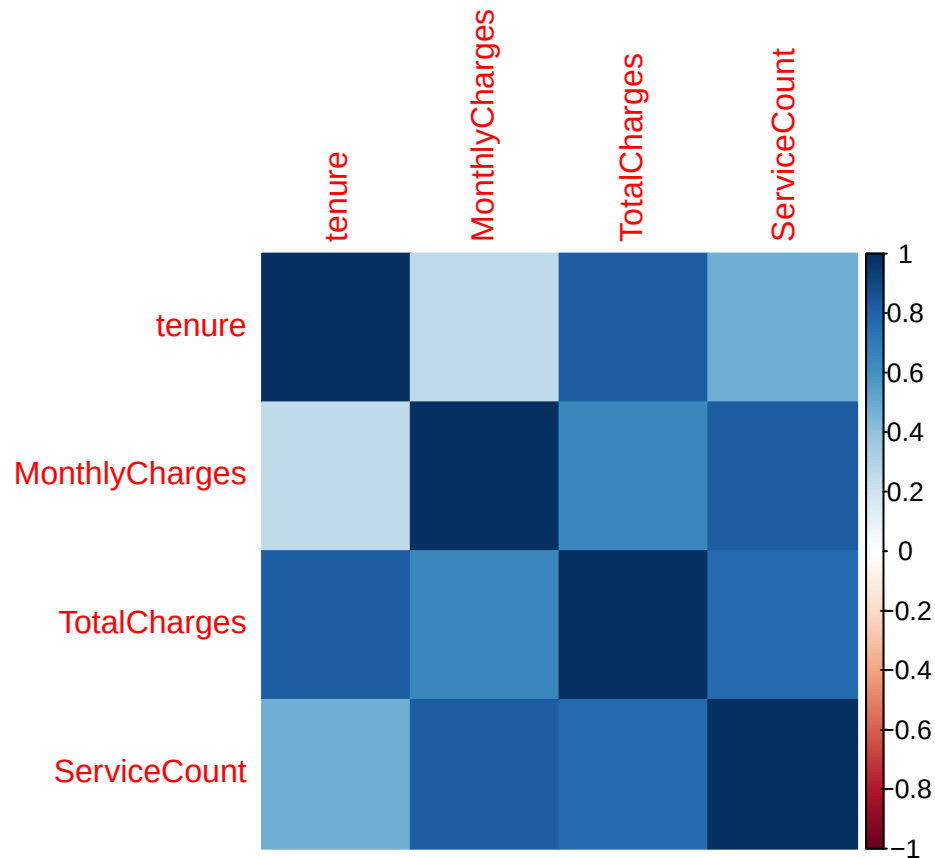
ADDRESSING OUTLIERS USING LOG TRANSFORMATIONS FOR TRAIN, VALID, TEST DATA

```
# TRAIN DATA  
# Applying the log1p transformation on TotalCharges variable for train, valid, test data  
train_data$TotalCharges <- log1p(train_data$TotalCharges)  
  
#summary(train_data$TotalCharges)  
  
# VALID DATA  
  
valid_data$TotalCharges <- log1p(valid_data$TotalCharges)  
  
# summary(tidy_df$LogTotalCharges)  
  
# TEST DATA  
  
test_data$TotalCharges <- log1p(test_data$TotalCharges)  
  
# summary(tidy_df$LogTotalCharges)
```

Handling the outliers or skewness ensures more accurate insights into feature/variable relationships and this helps correlation matrix will represent the relationships in the cleaned and processed data, which is closer to the data the model will use.

HEATMAP OF CORRELATIONS

```
cor_matrix <- cor(numerical_data)  
  
# The heatmap of correlations  
corrplot(cor_matrix, method = "color")
```

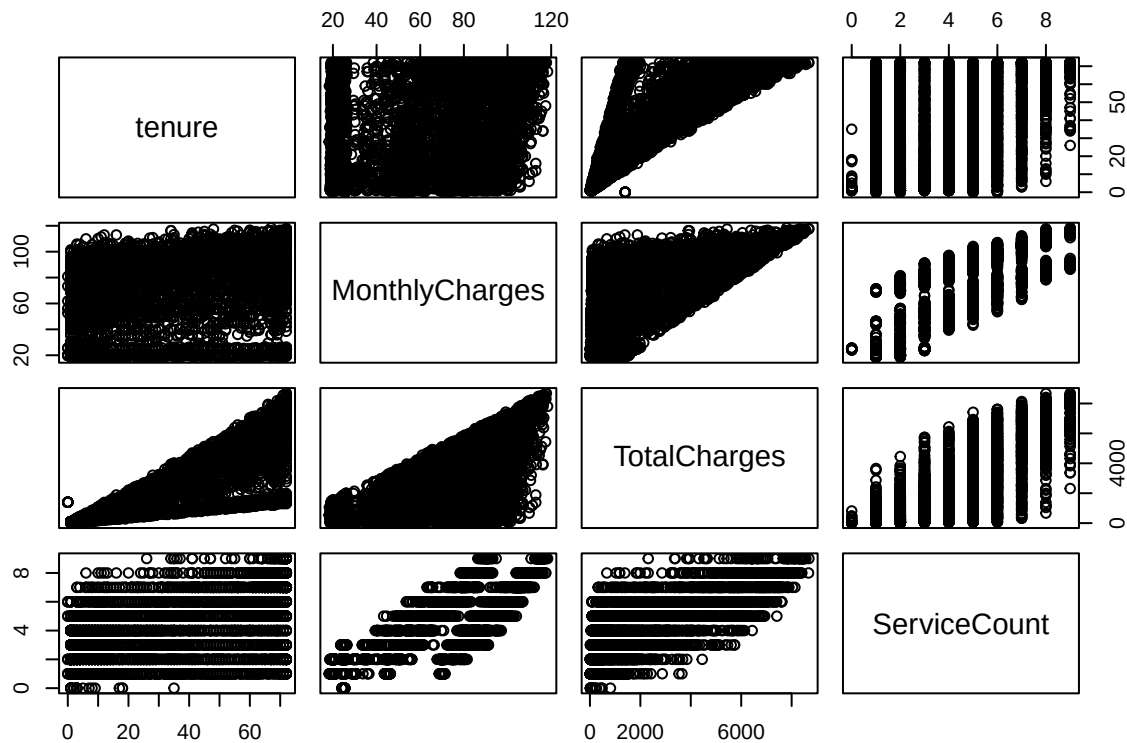



NUMERICAL DATA PAIRWISE PLOT

```
cor_matrix <- cor(numerical_data, use = "complete.obs")
cor_matrix
```

```
##           tenure MonthlyCharges TotalCharges ServiceCount
## tenure      1.0000000      0.2516703      0.8284083      0.4874862
## MonthlyCharges 0.2516703      1.0000000      0.6486581      0.8265249
## TotalCharges  0.8284083      0.6486581      1.0000000      0.7715763
## ServiceCount  0.4874862      0.8265249      0.7715763      1.0000000
```

```
pairs(numerical_data)
```



INTERPRETATION:

TotalCharges and tenure: 0.828 (Strong positive correlation)

- This high correlation is expected because `TotalCharges` accumulates over time (`tenure`). Therefore including both variables (`tenure` and `TotalCharges`) may lead to multicollinearity.

MonthlyCharges and ServiceCount: 0.827 (Strong positive correlation)

- This makes sense as customers with more services (`ServiceCount`) are likely to have higher `MonthlyCharges`. This relationship looks redundant and including both may not be significant.

TotalCharges and MonthlyCharges: 0.649 (Moderate positive correlation)

- `TotalCharges` depends partly on `MonthlyCharges` and `tenure`. However, `TotalCharges` has like cumulative effect.

ServiceCount and TotalCharges: 0.772 (Strong positive correlation)

- Customers with higher ServiceCount likely will have higher TotalCharges, and possible of having little redundancy/overlap information.

Where:

- *tenure*: loyalty and duration.
- *MonthlyCharges*: short term financial bills.
- *ServiceCount*: customer engagement in broad range of services.

FEATURE SELECTION:

Hence decided to drop **servicecount** and **totalcharges** variable because of high collinearity and we can also remove **customerID** as it doesn't have anything to do with churn prediction. This balance minimizes multicollinearity while retaining meaningful predictive features.

CATEGORICAL ANALYSIS

```
categorical_variables <- sapply(train_data, is.factor)
categorical_data <- train_data[, categorical_variables]

# The subsets 1, 2, and 3 combined equals categorical data;
# just for the convenience of displaying plots, the categorical data is split like this:

columns_subset1 <- c("gender", "Partner", "Dependents", "PhoneService", "MultipleLines", "Intern
columns_subset2 <- c("OnlineSecurity", "OnlineBackup", "DeviceProtection", "TechSupport", "Strea
columns_subset3 <- c("StreamingMovies", "Contract", "PaperlessBilling", "PaymentMethod", "Churn"

# Create the subsets using column indexing
categorical_data_1 <- categorical_data[, columns_subset1]
categorical_data_2 <- categorical_data[, columns_subset2]
categorical_data_3 <- categorical_data[, columns_subset3]
```

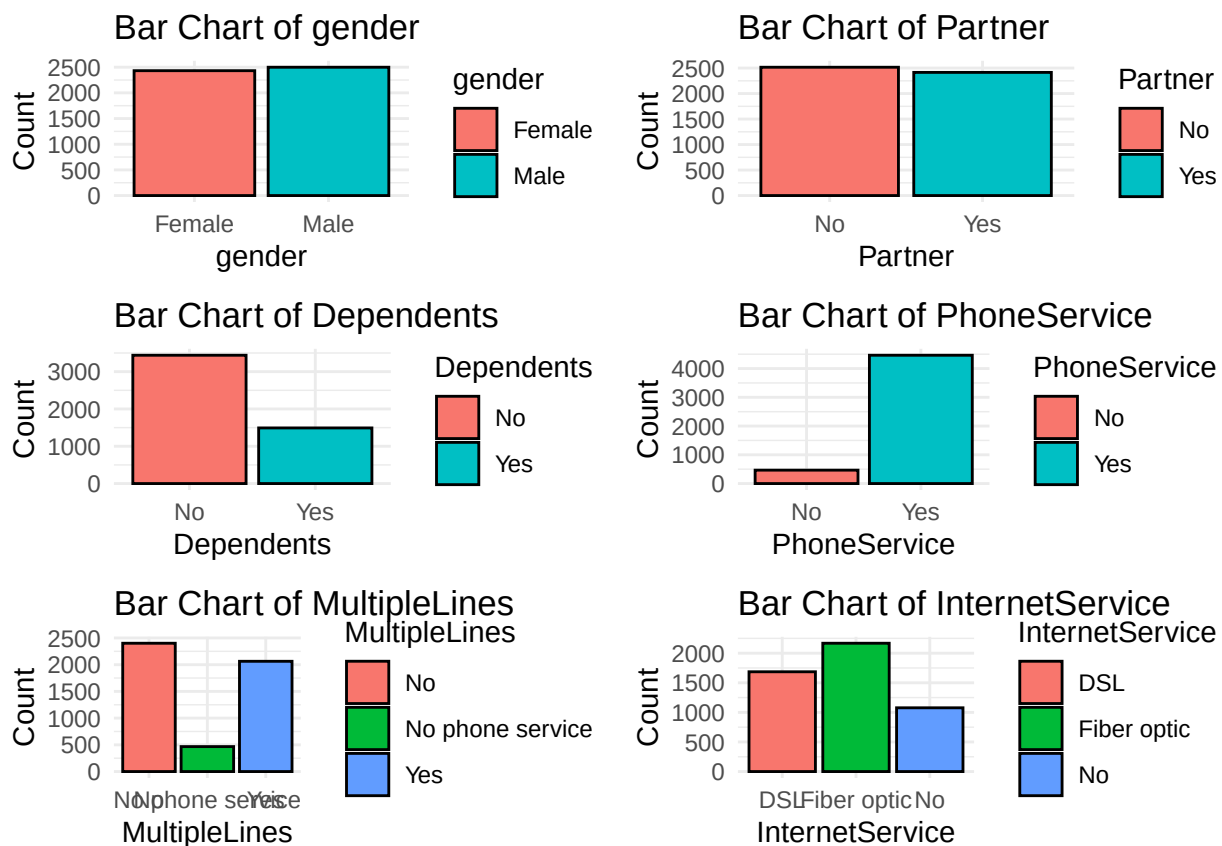
BAR CHARTS

```

#Using grid.arrange for categorical variables to plot many plots in a single window
# Create all bar plots and store them in a list
plot_list <- lapply(colnames(categorical_data_1), function(variable) {
  ggplot(categorical_data_1, aes(x = .data[[variable]], fill = .data[[variable]])) +
    geom_bar(color = "black") +
    labs(title = paste("Bar Chart of", variable), x = variable, y = "Count") +
    theme_minimal()
})

# Combine the plots into a grid layout
grid.arrange(grobs = plot_list, ncol = 2) # Adjust ncol for number of columns

```



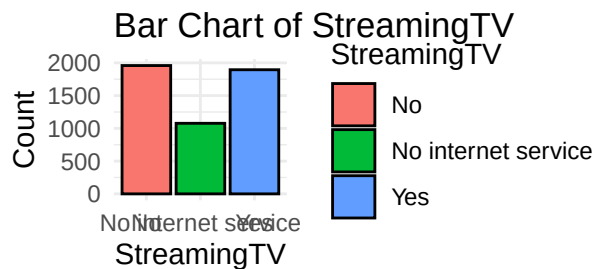
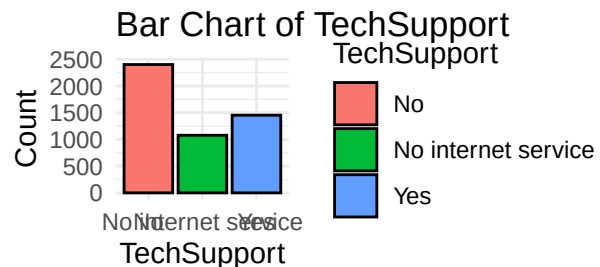
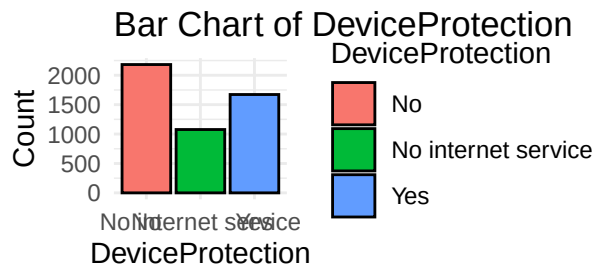
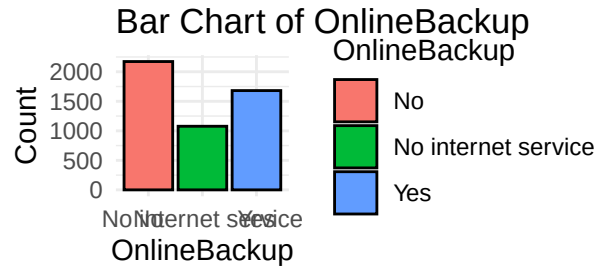
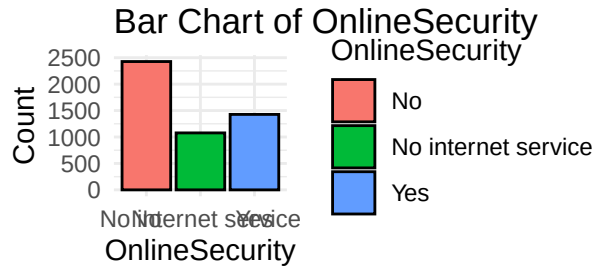
```

# Create all bar plots and store them in a list
plot_list <- lapply(colnames(categorical_data_2), function(variable) {
  ggplot(categorical_data_2, aes(x = .data[[variable]], fill = .data[[variable]])) +
    geom_bar(color = "black") +
    labs(title = paste("Bar Chart of", variable), x = variable, y = "Count") +
    theme_minimal()
})

# Combine the plots into a grid layout

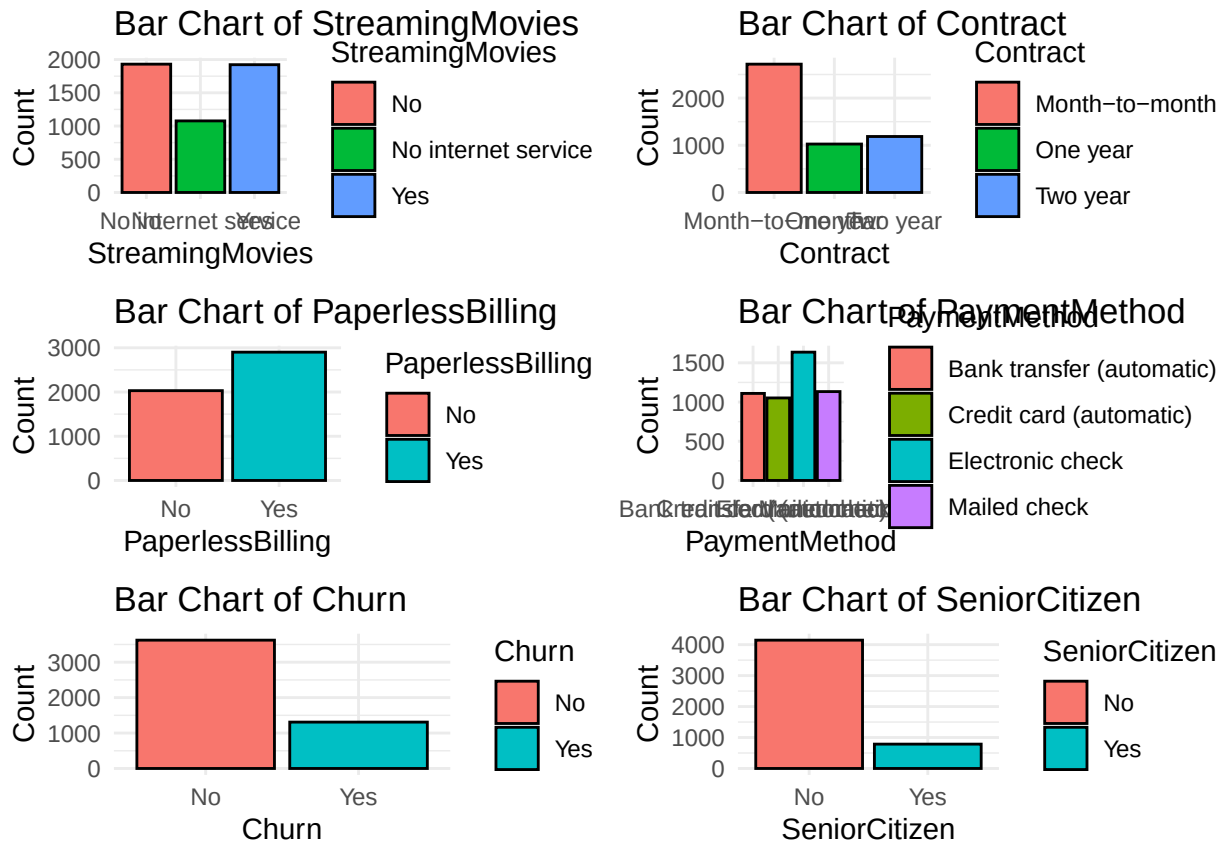
```

```
grid.arrange(grobs = plot_list, ncol = 2) # Adjust ncol for number of columns
```



```
# Create all bar plots and store them in a list
plot_list <- lapply(colnames(categorical_data_3), function(variable) {
  ggplot(categorical_data_3, aes(x = .data[[variable]], fill = .data[[variable]])) +
    geom_bar(color = "black") +
    labs(title = paste("Bar Chart of", variable), x = variable, y = "Count") +
    theme_minimal()
})

# Combine the plots into a grid layout
grid.arrange(grobs = plot_list, ncol = 2) # Adjust ncol for number of columns
```



INTERPRETATION:

- People count having Phonservices, Dependents are more compared to not.
- The gender and partner count looks almost similar for both possible classes.
- People mostly opt for montly charges.
- Very less seniorcitizen count, very less churn.
- People in this dataset represent has low importance on device protection, techsupport, on-line backup and online security.

BAR CHARTS WITH TARGET CHURN

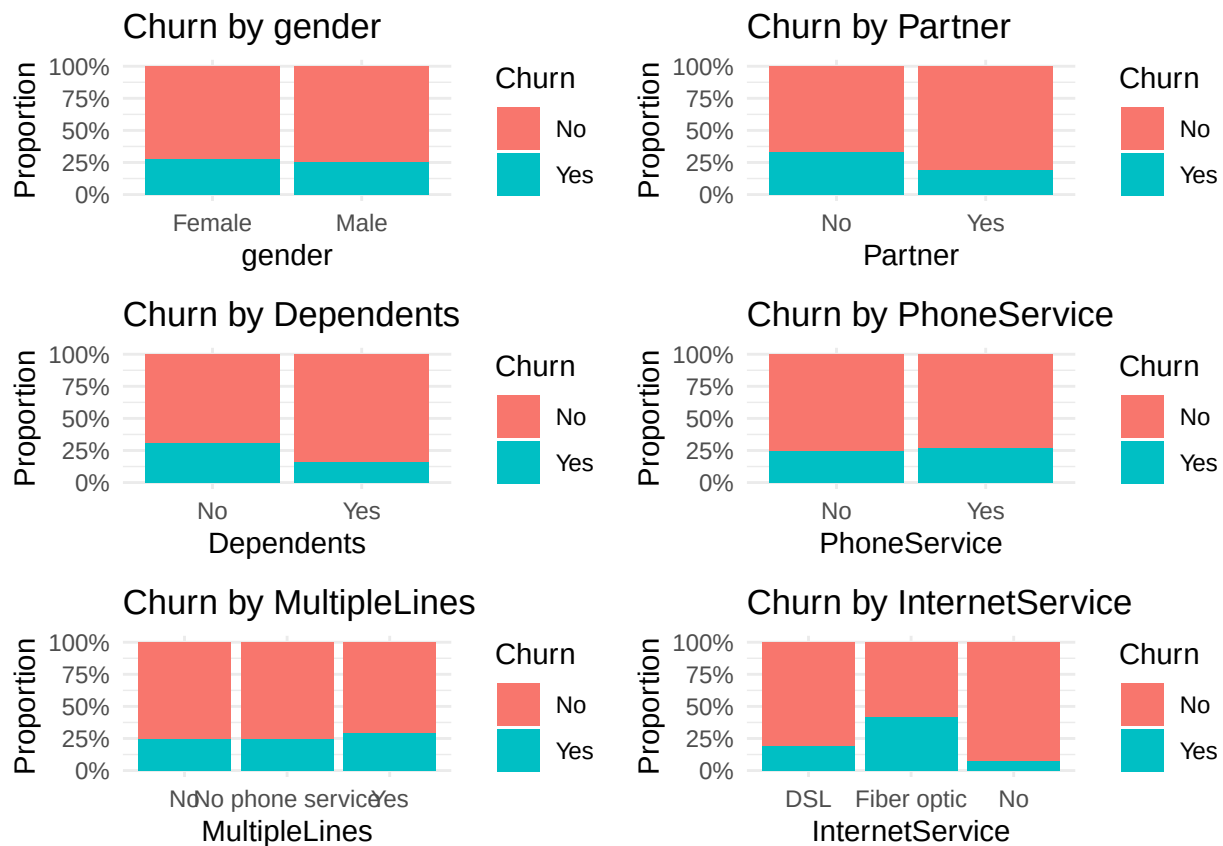
```
#fill="Churn" is added to the code so that the legened doesnt have weird name as categorical_data
# Create all bar plots and store them in a list
plot_list <- lapply(colnames(categorical_data_1), function(variable) {
  ggplot(categorical_data_1, aes(x = .data[[variable]], fill = categorical_data$Churn)) + # Fill
  geom_bar(position = "fill") +
  labs(title = paste("Churn by", variable), x = variable, y = "Proportion", fill = "Churn") +
```

```

theme_minimal() +
scale_y_continuous(labels = scales::percent)
})

# Combine the plots into a grid layout
grid.arrange(grobs = plot_list, ncol = 2) # Adjust ncol for number of columns

```

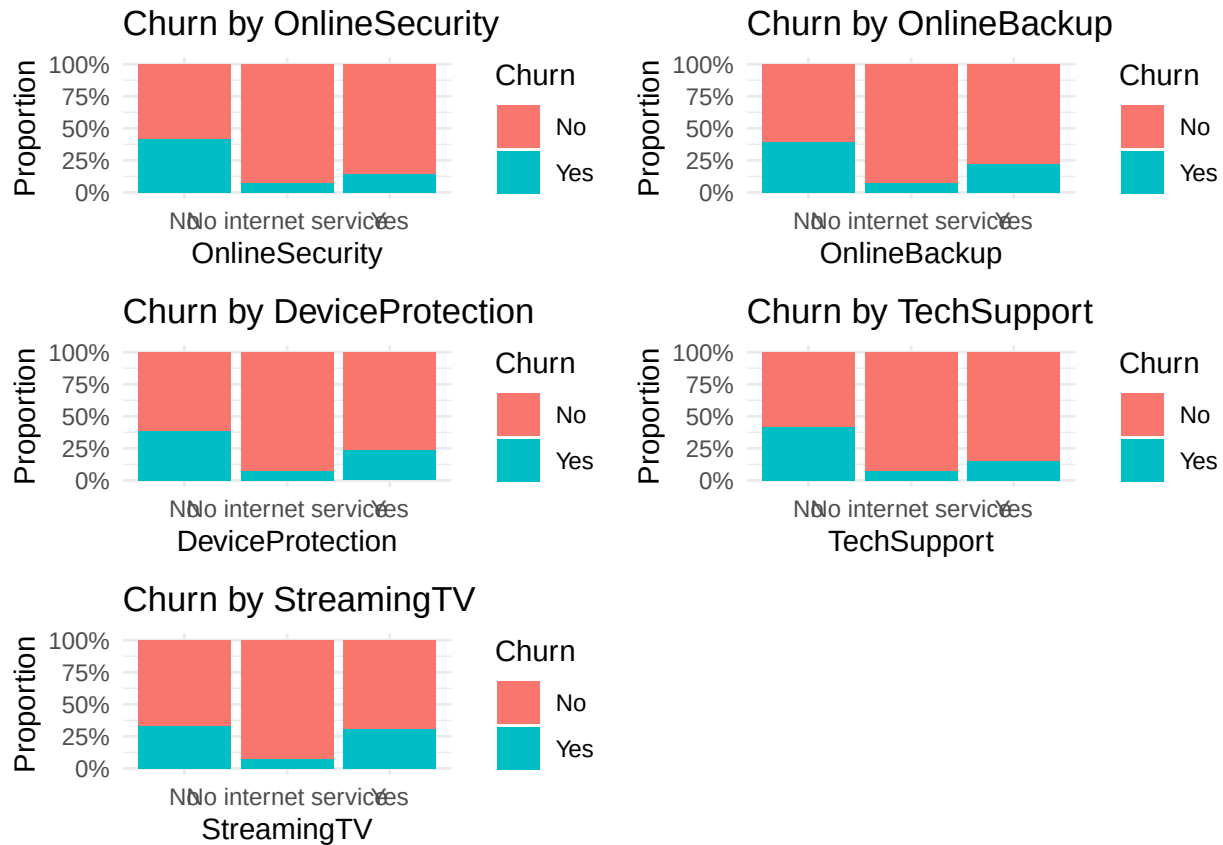


```

# Create all bar plots and store them in a list
plot_list <- lapply(colnames(categorical_data_2), function(variable) {
  ggplot(categorical_data_2, aes(x = .data[[variable]], fill = categorical_data$Churn)) + # Fill
  geom_bar(position = "fill") +
  labs(title = paste("Churn by", variable), x = variable, y = "Proportion", fill = "Churn") +
  theme_minimal() +
  scale_y_continuous(labels = scales::percent)
})

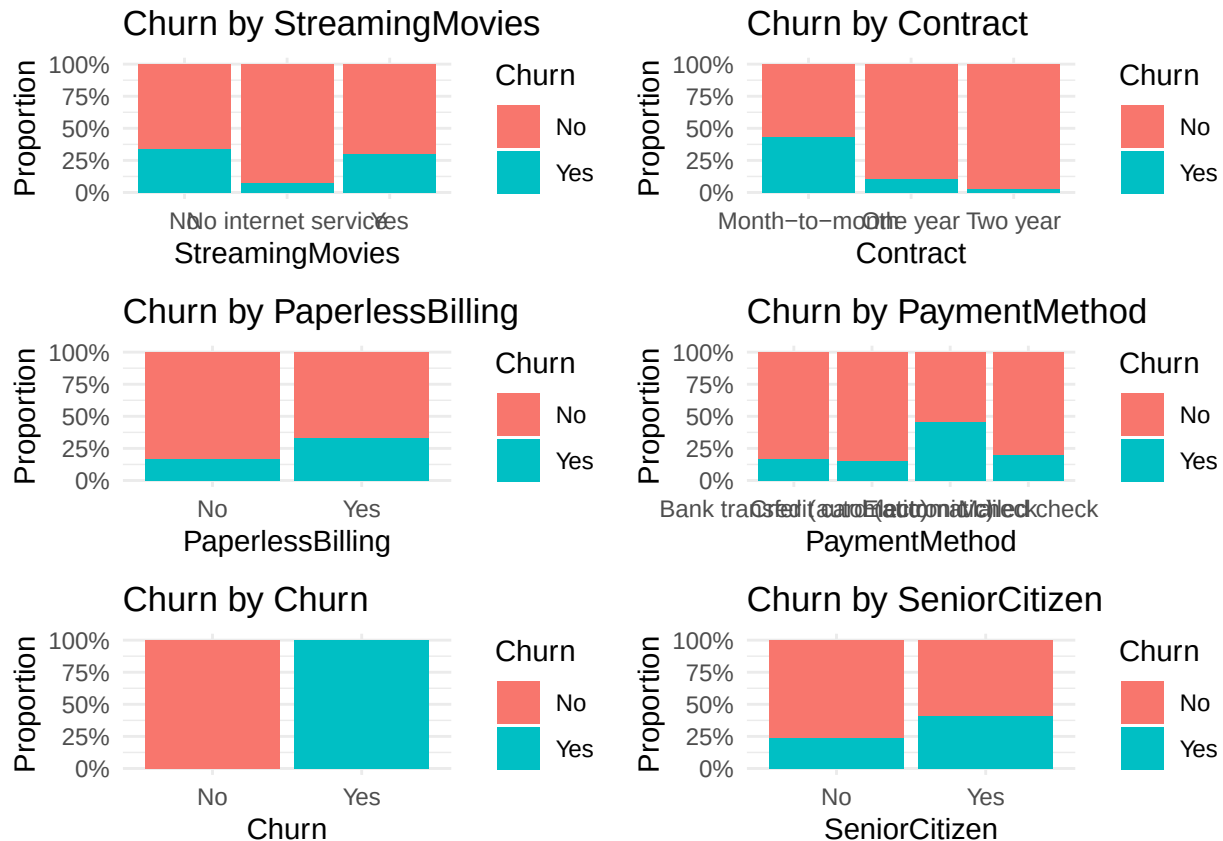
# Combine the plots into a grid layout
grid.arrange(grobs = plot_list, ncol = 2) # Adjust ncol for number of columns

```



```
# Create all bar plots and store them in a list
plot_list <- lapply(colnames(categorical_data_3), function(variable) {
  ggplot(categorical_data_3, aes(x = .data[[variable]], fill = Churn)) + # Fill by Churn
  geom_bar(position = "fill") +
  labs(title = paste("Churn by", variable), x = variable, y = "Proportion") +
  theme_minimal() +
  scale_y_continuous(labels = scales::percent)
})

# Combine the plots into a grid layout
grid.arrange(grobs = plot_list, ncol = 2) # Adjust ncol for number of columns
```

INTERPRETATION:

- People who has fibre optic as internet service opt out of service (churn) more than rest two.
- People churns more from the category of having no online security, online backup, techsupport, device protection.
- People churns more who has electronic check payment method than the rest.
- senior citizens opt to get out of service more, also monthly contract bills paying people churn more than the rest.

CROSS TABULATION AND CHI SQUARE TEST

Chi Square test to determine if there is a statistically significant association between two categorical variables and also between various categorical features and Churn like whether the customer has churned or not.

```
# Perform a chi-square test for independence between categorical features and churn
# Loop through each categorical feature
for (feature in colnames(categorical_data)) {
```

```

if (feature != "Churn") {
  # Cross tabulation
  cross_tab <- table(categorical_data[[feature]], categorical_data$Churn)

  # Chi-square test
  chi_sq_test <- chisq.test(cross_tab)

  # Check if p-value is greater than 0.05 (insignificant)
  if (chi_sq_test$p.value > 0.05) {
    cat(paste("Insignificant Feature:", feature, " ->p value:", round(chi_sq_test$p.value, 4),
  )
}
}
}

```

```

## Insignificant Feature: gender ->p value: 0.1233
## Insignificant Feature: PhoneService ->p value: 0.2488

```

INTERPRERATION:

Variables with p value < 0.05 (significant association with Churn) are:

- All the variables includes SeniorCitizen, Partner, Dependents, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod, tenure_bin have a significant relationship with customer churn, and these likely contribute to predicting churn.

Variables with p-value > 0.05 (no significant association with Churn) are:

- *gender*: p value = 0.1233 (no significant association).
- *PhoneService*: p value = 0.2488 (no significant association).

These above variables does not show a significant relationship with churn and thats why not useful for predicting churn and could be dropped.

Where:

- **Null Hypothesis H0:** No association between two variables(i.e.independent). Alternative Hypothesis H1: There is some association/relation between the two variables (i.e dependent).
- **p-value < 0.05:** Strong evidence against null hypothesis and can reject null hypothesis and conclude that there is a statistically significant association.
- **p-value > 0.05:** Weak evidence against null hypothesis and fail to reject null hypothesis and conclude that there is no significant association.

FEATURE SELECTION

Selecting only certain features to avoid multicollinearity and avoiding potential overfitting and reducing complexity for the models. Therefore in total from categorical and numerical features/variables the variables that can be dropped are: **totalcharges** , **servicecount**, **customerID**, **gender**, **phoneservice**.

```
train_data <- train_data[, !(names(train_data) %in% c("TotalCharges", "ServiceCount", "customerID", "gender", "phoneservice"))]
valid_data <- valid_data[, !(names(valid_data) %in% c("TotalCharges", "ServiceCount", "customerID", "gender", "phoneservice"))]
test_data <- test_data[, !(names(test_data) %in% c("TotalCharges", "ServiceCount", "customerID", "gender", "phoneservice"))]
```

ONE HOT ENCODING

Before performing encoding we must have to exclude the target Variable (Churn) from Encoding. The purpose of dummy encoding is to convert categorical features to numerical format way and can be fed to machine learning models. And only the input variables not target variable because it is dependent variable that can be used to predict.

Also encoding target variable along with features leads to data leakage and the machine learning models might learn patterns from target variable during training and eventually resulting in overfitting or misleading results.

```
# excluding target variable Churn
categorical_columns <- setdiff(names(train_data)[sapply(train_data, is.factor)], "Churn")

#creating dummy variable model for the categorical variables only
dummies <- dummyVars( ~ ., data = train_data[, categorical_columns], fullRank = TRUE)

# applying dummy variable model to categorical data
encoded_data <- predict(dummies, newdata = train_data[, categorical_columns])

# converting the encoded data to a data frame
encoded_data <- data.frame(encoded_data)

# Now lets combine encoded data with the original non categorical variables
final_train_data <- cbind(train_data[, !names(train_data) %in% categorical_columns], encoded_data)

# Adding the target variable back to final dataset
final_train_data$Churn <- train_data$Churn

# View the transformed dataset
#head(final_train_data, 3)
```

```

# VALID DATA
dummies <- dummyVars( ~ ., data = valid_data[, categorical_columns], fullRank = TRUE)

encoded_data <- predict(dummies, newdata = valid_data[, categorical_columns])

encoded_data <- data.frame(encoded_data)

final_valid_data <- cbind(valid_data[, !names(valid_data) %in% categorical_columns], encoded_data)

final_valid_data$Churn <- valid_data$Churn

#head(final_valid_data, 3)


# TEST DATA
dummies <- dummyVars( ~ ., data = test_data[, categorical_columns], fullRank = TRUE)

encoded_data <- predict(dummies, newdata = test_data[, categorical_columns])

encoded_data <- data.frame(encoded_data)

final_test_data <- cbind(test_data[, !names(test_data) %in% categorical_columns], encoded_data)

final_test_data$Churn <- test_data$Churn

#head(final_test_data, 3)

```

DROPPING THE VARIABLES WITH LOW VARIANCE AND HIGHLY CORRELATED AFTER ENCODING

```

# TRAIN DATA

```

```
# Checking for near-zero variance features and remove them
nzv <- nearZeroVar(final_train_data, saveMetrics = TRUE)
any(nzv$nzv)
```

```
## [1] FALSE
```

```
which(nzv$nzv)
```

```
## integer(0)
```

```
# Checking for highly correlated features (correlation > 0.9) and remove one from each pair
cor_matrix <- cor(final_train_data[, sapply(final_train_data, is.numeric)])
high_cor <- findCorrelation(cor_matrix, cutoff = 0.9) # Correlation threshold
final_train_data <- final_train_data[, -high_cor]
```

```
# VALID DATA
```

```
# Checking for near zero variance features and remove them
nzv <- nearZeroVar(final_valid_data, saveMetrics = TRUE)
any(nzv$nzv)
```

```
## [1] FALSE
```

```
which(nzv$nzv)
```

```
## integer(0)
```

```
cor_matrix <- cor(final_valid_data[, sapply(final_valid_data, is.numeric)])
high_cor <- findCorrelation(cor_matrix, cutoff = 0.9)
final_valid_data <- final_valid_data[, -high_cor]
```

```
# TEST DATA
```

```
# Checking for near zero variance features and remove them
nzv <- nearZeroVar(final_test_data, saveMetrics = TRUE)
any(nzv$nzv)
```

```
## [1] FALSE
```

```
which(nzv$nzv)
```

```
## integer(0)
```

```
cor_matrix <- cor(final_test_data[, sapply(final_test_data, is.numeric)])  
high_cor <- findCorrelation(cor_matrix, cutoff = 0.9)  
final_test_data <- final_test_data[, -high_cor]
```

```
train_data <- final_train_data
```

```
valid_data <- final_valid_data
```

```
test_data <- final_test_data
```

PREDICTIVE ANALYSIS AND RESULTS

Q1: Apply and document the statistical learning procedure for the predictive analysis.

The dataset is preprocessed (missing value imputation, feature creation, encoding categorical variables). After preprocessing, models like logistic regression, decision trees, or others can be trained on the `train_data`. The exact choice of statistical learning method will depend on the subsequent modeling phase.

Q2: Estimate the performance of the statistical learning approaches on test data, using resampling methods or other measures.

Performance can be assessed using metrics like accuracy, precision, recall, F1-score, and ROC-AUC. The model performance is estimated using resampling methods like cross-validation or by directly evaluating the model on the test data.

Q3: Evaluate the performance on the test data.

Evaluation is done by comparing predicted values against actual “Churn” values in the test set. Cross-tabulation (confusion matrix) and accuracy metrics like F1 score, ROC - AUC, etc will help assess the model’s performance.

Q4: Discuss the results.

The results will include performance metrics that are discussed above and the interpretation of those metrics. Key insights might performance of the best model on various metrics and its consistency on validation and test data and also including how well the model handles imbalance, and potential areas for improvement like tuning hyperparameters, improving feature selection and etc.

LOGISTIC REGRESSION MODEL

```
set.seed(123)
# Logistic Regression with 10-fold cross-validation
log_control <- trainControl(method = "cv", number = 10, summaryFunction = twoClassSummary, classProbs = "prob")
log_model <- train(Churn ~ ., data = train_data, method = "glm", family = "binomial", trControl = log_control)

# Model Training
log_model_final <- glm(Churn ~ ., data = train_data, family = "binomial")

# Model Evaluation Validation Data
log_pred_val <- predict(log_model_final, valid_data, type = "response")
log_pred_class <- ifelse(log_pred_val > 0.5, "Yes", "No")

# lapply(log_pred_class, levels)

# Convert Churn in validation data and predicted classes to factors
valid_data$Churn <- factor(valid_data$Churn, levels = c("No", "Yes"))
log_pred_class <- factor(log_pred_class, levels = c("No", "Yes"))

log_cm <- confusionMatrix(log_pred_class, valid_data$Churn)

# Calculating the metrics, Precision, Recall, F1-Score, and ROC-AUC for Logistic Regression
# and extract Precision, Recall, and F1-Score from confusionMatrix
log_precision <- log_cm$byClass["Pos Pred Value"] # Precision
log_recall <- log_cm$byClass["Sensitivity"] # Recall
log_f1 <- log_cm$byClass["F1"]
log_roc <- roc(valid_data$Churn, as.numeric(log_pred_val)) # Ensure Churn is numeric for ROC

log_precision

## Pos Pred Value
##      0.843562

log_recall

## Sensitivity
##      0.9033505
```

```
log_f1
```

```
##          F1  
## 0.8724331
```

```
log_roc
```

```
##  
## Call:  
## roc.default(response = valid_data$Churn, predictor = as.numeric(log_pred_val))  
##  
## Data: as.numeric(log_pred_val) in 776 controls (valid_data$Churn No) < 280 cases (valid_data$Churn Yes)  
## Area under the curve: 0.8441
```

RANDOM FOREST MODEL

```
set.seed(123)
```

```
# Random Forest with 10 fold cross-validation (default model, no hyperparameter tuning)  
rf_control <- trainControl(method = "cv", number = 10, summaryFunction = twoClassSummary, classProbs = "prob")  
rf_model <- train(Churn ~ ., data = train_data, method = "rf", trControl = rf_control, metric = "auc")
```

```
# MODEL EVALUATION
```

```
# Evaluating Random Forest on Validation Data
```

```
rf_pred_val <- predict(rf_model, valid_data)  
rf_cm <- confusionMatrix(rf_pred_val, valid_data$Churn)
```

```
# Calculating the metrics like Precision, Recall, F1-Score, and ROC-AUC for Random Forest  
# Extract Precision, Recall, and F1-Score from confusionMatrix for Random Forest
```

```
rf_precision <- rf_cm$byClass["Pos Pred Value"] # Precision  
rf_recall <- rf_cm$byClass["Sensitivity"] # Recall  
rf_f1 <- rf_cm$byClass["F1"] # F1-Score
```

```
# Calculate ROC AUC
```

```
rf_roc <- roc(valid_data$Churn, as.numeric(rf_pred_val)) # Ensure Churn is numeric for ROC
```

```
# Output the results
```

```
rf_precision
```

```
## Pos Pred Value  
##          0.7898627
```



```
rf_recall
```

```
## Sensitivity  
## 0.9639175
```

```
rf_f1
```

```
## F1  
## 0.868253
```

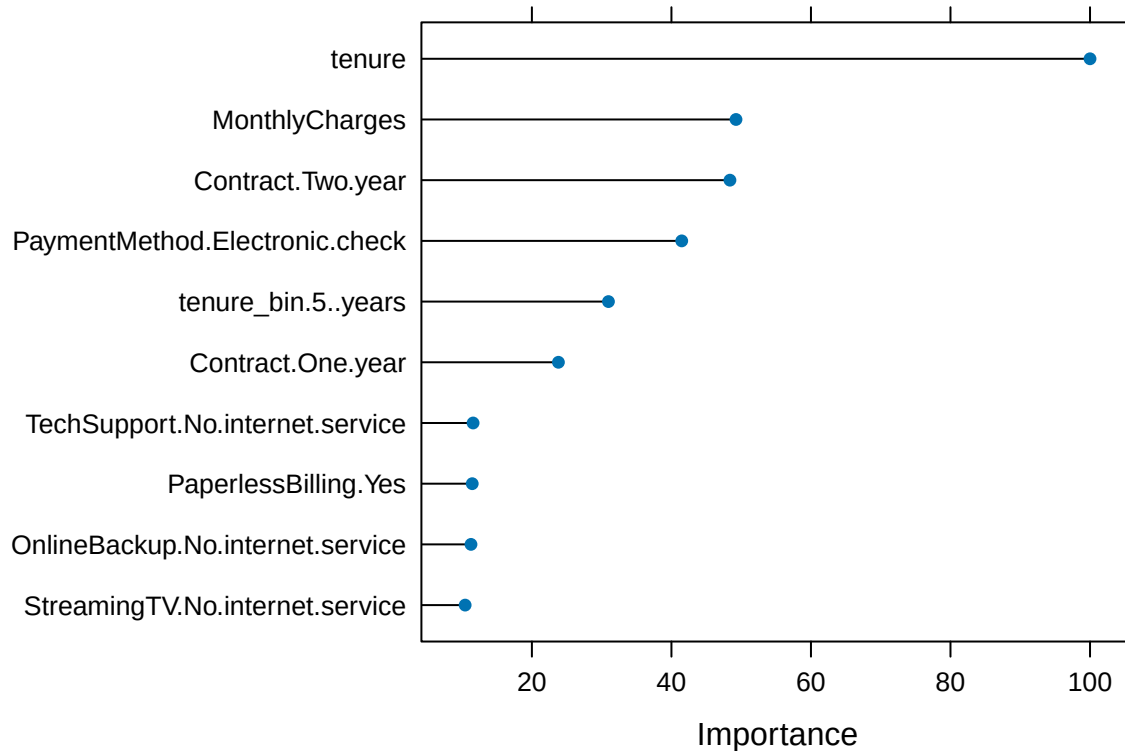
```
rf_roc
```

```
##  
## Call:  
## roc.default(response = valid_data$Churn, predictor = as.numeric(rf_pred_val))  
##  
## Data: as.numeric(rf_pred_val) in 776 controls (valid_data$Churn No) < 280 cases (valid_data$C  
## Area under the curve: 0.6266
```

```
rf_importance <- varImp(rf_model, scale = TRUE)
```

```
# rf_importance
```

```
plot(rf_importance, top = 10) # Adjust 'top' to display the desired number of features
```



The model is overly reliant on tenure variable

RANDOM FOREST MODEL WITH FINE TUNING

```
set.seed(123)

# grid for tuning nodesize and maxnodes as part of hyperparameter tuning
tune_grid <- expand.grid(
  mtry = c(2, 3, 4, 5, 6, 7),
  nodesize = c(1, 5, 10),
  maxnodes = c(10, 20, 30)
)

# 10 fold cross validation
rf_control <- trainControl(method = "cv", number = 10, summaryFunction = twoClassSummary, classP

# Fine tuned Random Forest and tuning hyperparameters
fine_tuned_rf_model <- function(train_data, valid_data, tune_grid) {
  # Store results of each grid search
  fine_tuned_rf_results <- list()
```

```

# iterating through each combination of mtry, nodesize, and maxnodes
for (i in 1:nrow(tune_grid)) {
  # Extract current combination of parameters
  params <- tune_grid[i, ]

  # Fitting fine tuned Random Forest model with current parameters
  fine_tuned_rf <- randomForest(
    Churn ~ .,
    data = train_data,
    mtry = params$mtry,
    nodesize = params$nodesize,
    maxnodes = params$maxnodes,
    importance = TRUE
  )

  # Predictions on validation data
  fine_tuned_rf_pred_val <- predict(fine_tuned_rf, valid_data)

  # Computing the confusion matrix
  fine_tuned_rf_cm <- confusionMatrix(fine_tuned_rf_pred_val, valid_data$Churn)

  # Extracting Precision, Recall, F1-Score, and ROC-AUC
  fine_tuned_rf_precision <- fine_tuned_rf_cm$byClass["Pos Pred Value"]
  fine_tuned_rf_recall <- fine_tuned_rf_cm$byClass["Sensitivity"]
  fine_tuned_rf_f1 <- fine_tuned_rf_cm$byClass["F1"]
  fine_tuned_rf_roc <- roc(valid_data$Churn, as.numeric(fine_tuned_rf_pred_val))

  # Saving result in the form of data frame
  fine_tuned_rf_results[[i]] <- data.frame(
    mtry = params$mtry,
    nodesize = params$nodesize,
    maxnodes = params$maxnodes,
    precision = fine_tuned_rf_precision,
    recall = fine_tuned_rf_recall,
    f1 = fine_tuned_rf_f1,
    roc_auc = auc(fine_tuned_rf_roc)
  )
}

# Combining all into one data frame
fine_tuned_rf_results_df <- do.call(rbind, fine_tuned_rf_results)

return(fine_tuned_rf_results_df)
}

```

```

# fine tuned Random Forest tuning on the train and validation data
fine_tuned_rf_results_df <- fine_tuned_rf_model(train_data, valid_data, tune_grid)

#print(fine_tuned_rf_results_df)

# Fetcing the best model based on ROC AUC
best_fine_tuned_rf_model <- fine_tuned_rf_results_df[which.max(fine_tuned_rf_results_df$roc_auc)]
print("Best Fine-Tuned Random Forest Model Parameters:")

```

```
## [1] "Best Fine-Tuned Random Forest Model Parameters:"
```

```
print(best_fine_tuned_rf_model)
```

```
##               mtry nodesize maxnodes precision    recall      f1    roc_auc
## Pos Pred Value53      7         10         30 0.8204545 0.9304124 0.8719807 0.6830633
```

XGBOOST MODEL

```

set.seed(123)

# Converting Churn column target variabe to numeric (0 = No, 1 = Yes)
train_data$Churn <- ifelse(train_data$Churn == "Yes", 1, 0)
valid_data$Churn <- ifelse(valid_data$Churn == "Yes", 1, 0)

# Extracting features X and target y
X_train <- as.matrix(train_data[, -which(names(train_data) == "Churn")]) # Remove target variab
y_train <- train_data$Churn

X_valid <- as.matrix(valid_data[, -which(names(valid_data) == "Churn")]) # Remove target variab
y_valid <- valid_data$Churn

# 10 Fold Cross Validation using internal cv not from exeteernal library
xgb_cv_results <- xgb.cv(
  data = X_train,
  label = y_train,
  nfold = 10,                                # Number of folds
  nrounds = 1000,                            # Number of boosting rounds
  objective = "binary:logistic",             # Binary classification

```

```

    eval_metric = "logloss",      # Evaluation metric
    max_depth = 6,                # Maximum depth of trees
    eta = 0.01,                  # Learning rate
    gamma = 0,                   # Minimum loss reduction
    colsample_bytree = 0.8,       # Fraction of columns sampled per tree
    subsample = 0.8,             # Fraction of rows sampled
    scale_pos_weight = 3,        # Class imbalance adjustment
    verbose = 0                  # Suppress verbose output
)

# Train XGBoost Model
xgb_model <- xgboost(
  data = X_train,
  label = y_train,
  nrounds = 1000,               # Number of boosting rounds
  objective = "binary:logistic", # Binary classification (churn / not churn)
  eval_metric = "logloss",      # Evaluation metric (logloss)
  max_depth = 6,               # Maximum depth of each tree
  eta = 0.01,                  # Learning rate
  gamma = 0,                   # Minimum loss reduction required to make a further partition
  colsample_bytree = 0.8,       # Fraction of columns to sample per tree
  subsample = 0.8,             # Fraction of rows to sample
  scale_pos_weight = 3,        # Class imbalance handling
  verbose = 0                  # to suppress output
)

# Model Evaluation on Validation Data
xgb_pred_val <- predict(xgb_model, X_valid)
xgb_pred_class_val <- ifelse(xgb_pred_val > 0.5, 1, 0) # Convert to binary class (1 = churn, 0

# Confusion Matrix
xgb_cm_val <- confusionMatrix(as.factor(xgb_pred_class_val), as.factor(y_valid))
print(xgb_cm_val)

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 581  60
##           1 195 220
##
##           Accuracy : 0.7585
##           95% CI : (0.7315, 0.7841)
##           No Information Rate : 0.7348

```

```
##      P-Value [Acc > NIR] : 0.0428
##
##              Kappa : 0.4631
##
## McNemar's Test P-Value : <2e-16
##
##              Sensitivity : 0.7487
##              Specificity : 0.7857
##              Pos Pred Value : 0.9064
##              Neg Pred Value : 0.5301
##              Prevalence : 0.7348
##              Detection Rate : 0.5502
##      Detection Prevalence : 0.6070
##      Balanced Accuracy : 0.7672
##
##      'Positive' Class : 0
##
```

```
# Precision, Recall, F1-Score, and ROC AUC for Validation Data
xgb_precision_val <- xgb_cm_val$byClass["Pos Pred Value"]
xgb_recall_val <- xgb_cm_val$byClass["Sensitivity"]
xgb_f1_val <- xgb_cm_val$byClass["F1"]
xgb_roc_val <- roc(y_valid, xgb_pred_val) # ROC-AUC
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
print(xgb_precision_val)
```

```
## Pos Pred Value
##      0.9063963
```

```
print(xgb_recall_val)
```

```
## Sensitivity
##      0.7487113
```

```
print(xgb_f1_val)
```

```
##      F1
## 0.8200423
```

```
print(paste("ROC-AUC: ", auc(xgb_roc_val)))
```

```
## [1] "ROC-AUC: 0.839110824742268"
```

MODEL SELECTION

```
# The logistic regression ROC-AUC is `r log_roc`  
# The random forest ROC-AUC is `r rf_roc`  
print("The logistic regression ROC AUC:0.8441")
```

```
## [1] "The logistic regression ROC AUC:0.8441"
```

```
print("The random Forest ROC Auc: 0.6266")
```

```
## [1] "The random Forest ROC Auc: 0.6266"
```

```
print(paste("fine tuned random forest ROC-AUC:",best_fine_tuned_rf_model$roc_auc))
```

```
## [1] "fine tuned random forest ROC-AUC: 0.683063328424153"
```

```
print(paste("xgboost model recall:",xgb_recall_val))
```

```
## [1] "xgboost model recall: 0.748711340206186"
```

```
print(paste(" xgBoost ROC-AUC: ", auc(xgb_roc_val)))
```

```
## [1] " xgBoost ROC-AUC: 0.839110824742268"
```

LOGISTIC REGRESSION AS FINAL MODEL:

It has the highest ROC AUC (0.8441) and also has balanced performance with Good scores across precision, recall, F1-score, and ROC AUC. It achieves a high recall, meaning it effectively identifies churners.

Its interpretability and simplicity make it easier to communicate the model's predictions and impact to stakeholders.

Most consistent performance, Less complex model, Robust against overfitting, Good balance of precision and recall, Easy Interpretable, Performs well on imbalanced data.

RANDOM FOREST MODELS:

These models are poorly performed compared to logistic regression and XGboost model.

Default Random Forest: Low ROC AUC (0.6266) makes it unsuitable, despite its high recall value.

Fine-Tuned Random Forest: Improved metrics but still low ROC AUC (0.6831), which is critical for evaluating the overall quality of predictions.

XGBoost:

While XGBoost has high precision and strong ROC AUC (0.8391), its lower recall (0.7487) means it may miss actual churners, which is risky for a class imbalanced problem. Additionally, XGBoost's added complexity makes it less interpretable.

When simple model and complex model both have similar performances choosing the simple model is suggested so Logistic Regression Model is chosen for now

FINAL TESTING

LOGISTIC REGRESSION ON TEST DATA

```
# Predictions on Test Data using Logistic Regression
log_pred_test <- predict(log_model_final, test_data, type = "response")
log_pred_class_test <- ifelse(log_pred_test > 0.5, "Yes", "No")

# Converting Churn in test data and predicted classes to factors
test_data$Churn <- factor(test_data$Churn, levels = c("No", "Yes"))
log_pred_class_test <- factor(log_pred_class_test, levels = c("No", "Yes"))

# Confusion Matrix
log_cm_test <- confusionMatrix(log_pred_class_test, test_data$Churn)

# Extract Precision, Recall, F1-Score, and ROC-AUC for Logistic Regression
log_precision_test <- log_cm_test$byClass["Pos Pred Value"] # Precision
log_recall_test <- log_cm_test$byClass["Sensitivity"] # Recall
log_f1_test <- log_cm_test$byClass["F1"]
log_roc_test <- roc(test_data$Churn, as.numeric(log_pred_test)) # ROC-AUC
```

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls < cases
```

```
log_precision_test
```



```
## Pos Pred Value
##      0.8385167
```

```
log_recall_test
```

```
## Sensitivity
##      0.9033505
```

```
log_f1_test
```

```
##      F1
## 0.869727
```

```
log_roc_test
```

```
##
## Call:
## roc.default(response = test_data$Churn, predictor = as.numeric(log_pred_test))
##
## Data: as.numeric(log_pred_test) in 776 controls (test_data$Churn No) < 280 cases (test_data$Churn Yes)
## Area under the curve: 0.8377
```

On the test dataset the logistic regression maintained strong performance. Ensures most of the predicted churn cases are accurate, High sensitivity confirms effective identification of actual churners.

High F1 score balance well between precision and recall. The good ROC AUC number indicates robust discriminative capability.

These results validate the choice of logistic regression as a model that balances predictive accuracy, interpretability, and simplicity, making it highly effective for the churn prediction task.

Therefore choosing logistic regression as the final model was a good decision for now. Its balanced performance on key metrics, ease of interpretation, robustness, and reliability on both validation and test datasets demonstrate that it is a practical and effective solution for predicting customer churn and consistently performed on validation and test data.

CONCLUSION:

Q1: The scope and generalizability of the predictive analysis.

SCOPE:

This predictive analysis is focused on identifying churners of telecommunications domain but can be generalized to similar industries where customer churn dynamics are comparable like digital media platforms like netflix, amazonPrime, ect like subscription based services.

The resulted high recall ensures that most churners are identified and thereby performs better customer outreach campaigns.

GENERALIZABILITY:

Logistic regression model's simplicity and robustness reduce overfitting, making it potentially generalizable to other datasets with similar patterns of imbalance and categorical variables.

Also maintains consistency in metrics across validation and test datasets so good reliability on unseen data.

Q2: Potential limitations and possibilities for improvement.

LIMITATIONS:

Linear Assumption: Logistic regression assumes the relation linearly which struggles badly while introduced to non linear data.

Data Imbalance:

- Logistic regression performed well with imbalanced data but has lower precision compared to recall so class imbalance could make bias predictions.
- Categorical are encoded by one hot encoding, which may lead to higher dimensional data. And this approach might not fully capture complex relationships.
- Potentially when required the important interactions between features might be underrepresented in a linear model like logistic regression.

Limited Model Complexity:

- Logistic regression assumes linear relationships between predictors and the target variable, which may not fully capture complex, non linear churn behaviors.
- Ensemble models like Random Forest or XGBoost might excel in identifying such complexities, though they currently performed poor for this project.

POSSIBILITIES FOR IMPROVEMENTS

Advanced Feature Engineering:

- Incorporate feature interactions, polynomial terms might provide still more better performance for all the models.

Handling Class Imbalance

- Using advanced techniques like Synthetic Minority Oversampling Technique (SMOTE) to balance the classes further during training. OR using stratified K fold cross validation during training.

Better Fine Tuning

- Random Forest and XGBOOST might have better performed over logistic regression if they were perfectly tuned in an optimised way that could provide the best result out of it which requires keen observation in depth knowledge on domain and much experience in fine tuning the models.
- Which might include use of right number of mtry, nodes, weights and etc by perfectly fine tuning
- By addressing these limitations and leveraging potential improvements, the predictive analysis can be extended and refined to offer even greater value in predicting the churners in telecommunications domain and also across different contexts.